SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

*I **J.Mohamed Aseem(192511258)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **J.MOHAMED ASEEM**

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

1. Project Vision and Stakeholder Identification

1.1 Project Vision

The vision of the Intelligent Software Requirements Traceability Platform (ISRT Platform) is to provide a centralized web-based system that connects software requirements with design artifacts, source-code modules, test cases, defects, and software releases. The platform will reduce manual traceability work, make project changes easier to analyse, improve test coverage, and help teams deliver reliable software.

1.2 Project Objectives

- Create, update, categorize, and manage software requirements in one repository.
- Establish traceability links between requirements, design documents, implementation components, and test cases.
- Record defects and connect them to affected requirements and test cases.
- Generate forward and backward traceability matrices.
- Identify missing links, untested requirements, and requirements affected by proposed changes.
- Provide clear reports that support project monitoring and software quality improvement.

1.3 Key Stakeholders and Their Roles

Stakeholder	Role	Main Need
Product Owner / Client	Defines business needs and prioritizes requirements.	Visibility of requirement status and business value.
Business Analyst	Documents requirements and maintains links to design.	Accurate, complete, changeable requirements.
Project Manager / Scrum Master	Plans sprints and monitors progress and risks.	Progress reports, traceability gaps, and impact summaries.
Software Developer	Links source-code modules and implementation changes to requirements.	Know which requirements and components are affected.
QA / Test Engineer	Creates test cases, records execution, and maps tests to requirements.	Coverage reports and identification of untested requirements.
Defect / Support Engineer	Logs defects and links them to related requirements and tests.	Defect status and affected components.
Release Manager	Tracks which requirements and fixes are included in a release.	Release readiness and change impact.
System Administrator	Manages user accounts, roles, and access permissions.	Secure and controlled platform access.

2. Epics and User Stories

The project is divided into epics. Each user story follows the Agile format: “As a [user], I want [feature], so that [benefit].”

Epic	ID	User Story (Summary)	Priority	Points
E1 – Requirement Management	US-01	As a business analyst, I want to create and edit requirements with unique IDs and priorities, so that requirements are managed consistently.	High	3
E2 – Traceability Linking	US-02	As a developer, I want to link requirements to design artifacts and source-code modules, so that implementation can be traced back to business needs.	High	5
E3 – Test Coverage	US-03	As a QA engineer, I want to link test cases to requirements, so that I can verify that every requirement is tested.	High	5
E4 – Defect Tracking	US-04	As a QA engineer, I want to record defects and link them to related requirements and test cases, so that	Medium	3

		defects can be investigated in context.		
E5 – Traceability Matrix & Reports	US-05	As a project manager, I want to generate a traceability matrix, so that I can identify missing links and review coverage.	High	5
E6 – Change Impact Analysis	US-06	As a project manager, I want to view linked artifacts when a requirement changes, so that the team can estimate the impact before implementation.	Medium	8
E7 – Access Control	US-07	As an administrator, I want to assign roles to users, so that project information is accessed only by authorized people.	Medium	3
E8 – Release Tracking	US-08	As a release manager, I want to associate requirements and defects with a release, so that I can check release completeness.	Low	5

3. Acceptance Criteria

US-01 – Requirement Management

- Given an authorized analyst is signed in, when valid requirement details are submitted, then the system saves the requirement with a unique ID.
- Given a requirement exists, when the analyst edits its description or priority, then the updated details are saved and displayed.
- Given mandatory fields are missing, when the analyst submits the form, then the system shows validation messages and does not save incomplete data.

US-02 – Traceability Linking

- Given a requirement and a design artifact or code module exist, when the developer creates a link, then the relationship is stored and visible from both records.
- Given a link already exists, when the user attempts to create the same link again, then the system prevents or clearly flags the duplicate.
- Given an artifact is opened, when its traceability links are viewed, then linked requirements and artifacts are listed.

US-03 – Test Coverage

- Given a requirement and test case exist, when the QA engineer links them, then the link appears in the requirement and test-case views.
- Given a requirement has no linked test case, when coverage is generated, then that requirement is identified as not covered.
- Given linked test cases have execution statuses, when coverage is viewed, then the report displays the relevant status.

US-04 – Defect Tracking

- Given a defect is submitted with a title, description, and severity, when it is saved, then it receives a unique defect ID.
- Given a defect is related to a requirement or test case, when the user adds the link, then the relationship is displayed in both records.
- Given a defect status changes, when the record is updated, then the latest status and update details are shown.

US-05 – Traceability Matrix & Reports

- Given traceability links exist, when an authorized user requests a matrix, then the system displays requirements against linked design, code, tests, and defects.
- Given a requirement has a missing link in a selected category, when the matrix is generated, then the gap is clearly marked.
- Given a matrix is displayed, when the user filters by priority or status, then only matching records are shown.

US-06 – Change Impact Analysis

- Given a requirement has linked artifacts, when the requirement is marked as changed, then the system lists directly linked design, code, test, defect, and release records.
- Given no links exist for a requirement, when impact analysis is run, then the system reports that no linked artifacts were found and flags a possible traceability gap.

US-07 – Access Control

- Given an administrator assigns a role, when a user signs in, then available actions match the permissions for that role.
- Given a user lacks edit permission, when the user attempts to modify a requirement, then the system denies the action and displays an appropriate message.

US-08 – Release Tracking

- Given requirements and defects are linked to a release, when the release summary is opened, then associated items and their statuses are displayed.
- Given a release contains unresolved high-severity defects, when readiness is reviewed, then those defects are highlighted for review.

4. Product Backlog Prioritization

The backlog is prioritized using business value, dependencies, risk, and the need to demonstrate a working minimum viable product (MVP). “High” items are planned earlier. Story points use a relative Fibonacci-style scale.

Rank	Story ID	Backlog Item	Priority	Story Points
1	US-01	Requirement creation and editing	High	3
2	US-02	Link requirements to design/code	High	5
3	US-03	Map test cases to requirements	High	5
4	US-05	Generate traceability matrix	High	5
5	US-04	Defect tracking and linking	Medium	3
6	US-07	Role-based access control	Medium	3
7	US-06	Change impact analysis	Medium	8
8	US-08	Release tracking	Low	5

Dependency rationale: requirement management must exist before links can be created. Traceability links are needed before a meaningful matrix can be generated. Test mapping depends on requirements and test-case records. Advanced impact analysis and release tracking are scheduled after the core traceability workflow is working.

5. Sprint Backlog and Task Breakdown

Sprint 1 Plan

Duration: 2 weeks (10 working days) | Team capacity assumption: approximately 16 story points

Selected user stories: US-01 (3 points), US-02 (5 points), and US-03 (5 points). Total = 13 story points. This leaves a small capacity buffer for integration, review, and unexpected work.

Story	**Task**	**Responsible Role**	**Estimate**
US-01	Confirm requirement fields and validation rules	Business Analyst / Developer	1 day
US-01	Design requirement entry and list screens	UI Developer	1 day
US-01	Implement create, edit, unique ID, and validation	Developer	2 days
US-01	Test requirement create/edit and validation	QA Engineer	1 day
US-02	Define traceability relationship data model	Developer	1 day
US-02	Build linking interface for requirement/design/code	Developer	2 days
US-02	Test link creation, viewing, and duplicate handling	QA Engineer	1 day
US-03	Define test-case fields and statuses	QA Engineer / Analyst	1 day
US-03	Implement requirement-to-test-case mapping	Developer	2 days
US-03	Show basic coverage status and test integration	Developer / QA	1 day
All	Sprint review, bug fixing, and demonstration	Whole team	1 day

6. Story Point Estimation and Justification

Technique: Planning Poker with a Fibonacci-style sequence (1, 2, 3, 5, 8, 13). The team compares relative complexity, implementation effort, dependencies, and uncertainty. Story points are estimates, not hours.

Story	**Points**	**Reason for Estimate**	**Sprint Decision**
US-01	3	Straightforward forms, data validation, and basic create/edit operations.	Sprint 1
US-02	5	Requires relationship data modelling, UI changes, and link integrity checks.	Sprint 1
US-03	5	Requires test-case mapping, coverage status, and integration testing.	Sprint 1
US-04	3	Basic defect records and links; moderate validation effort.	Later sprint
US-05	5	Matrix generation and filtering require aggregation of traceability links.	Later sprint
US-06	8	Must traverse multiple linked artifact types and handle missing links and change scenarios.	Later sprint
US-07	3	Role definitions and permission checks across common actions.	Later sprint
US-08	5	Release association and summary views across requirements and defects.	Later sprint

7. Sprint Goal and Expected Deliverables

7.1 Sprint Goal

By the end of Sprint 1, deliver a usable foundation for the Intelligent Software Requirements Traceability Platform that allows authorized users to manage requirements, link them to design/code artifacts, and associate test cases with requirements.

7.2 Expected Deliverables

- A working requirement management screen with create/edit functionality, unique IDs, and validation.
- A traceability-link feature connecting requirements to design artifacts and source-code modules.
- A test-case mapping feature connecting requirements to test cases.
- A basic coverage view that identifies requirements with and without linked tests.
- Test evidence for the selected user stories, including validation, link integrity, and mapping tests.
- A sprint review demonstration and an updated product backlog for Sprint 2.

7.3 Definition of Done

- Acceptance criteria for the story are met and demonstrated.
- Code is reviewed and merged into the shared project branch.
- Relevant unit and integration tests pass.
- No known critical defect remains in the completed functionality.
- User-facing fields and error messages are understandable.
- The product owner or assigned reviewer accepts the completed story.

8. Expected Project Outcomes

- Improved visibility of requirements and their related software artifacts.
- Better test coverage through explicit requirement-to-test mappings.
- Reduced traceability gaps by highlighting missing relationships.
- Easier impact analysis when requirements change.
- Improved collaboration among analysts, developers, testers, and release managers.
- Better software quality through organized evidence and defect tracking.

9. Conclusion

The Intelligent Software Requirements Traceability Platform applies Agile practices by organizing project work into epics, user stories, measurable acceptance criteria, a prioritized product backlog, and a realistic first sprint. Sprint 1 focuses on the foundation: managing requirements, creating traceability links, and mapping tests. Later sprints can add defect tracking, matrix reports, role-based access, impact analysis, and release tracking. This incremental approach supports early feedback, visible progress, and continuous improvement in software quality.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25